

# **FITNESSSTUDIO- DATENBANK**

Vorgetragen am 19.02.2026

David Seidl (10246056)  
Timur Scholl (10248102)  
Silas Siragusano (10245154)  
Din Bilalic (10249703)  
Jan Lauterbach (10244693)

# Gliederung

Block 1 (David): Grundlagen und Anforderungen der DB

Block 2 (Timur): Konzeptionelles Modell (ERD)

Block 3 (Silas): Physikalische Datenmodellierung

Block 4 (Jan): Implementierung in MySQL

Block 5 (Din): Datenauswertung und Transaktionscases mit SQL

# 01

## **GRUNDLAGEN UND ANFORDERUNGEN DER DATENBANK**

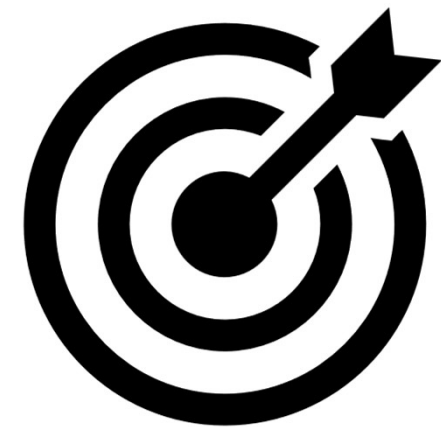
# WARUM EINE DATENBANK IM FITNESSSTUDIO ?

1. Viele verschiedene Informationen  
-> Zentrale Datenspeicherung
2. Gleichzeitiger Zugriff auf die DB  
z.B. Fall von mehreren Neuanmeldungen
3. Täglich neue Einträge  
-> Schnelle Datenänderung
4. Zusammenhängende Daten  
-> z. B. Mitgliedsdaten und Check In Zeiten



# FOKUS DER DATENBANK

1. Konsistente Datenspeicherung  
-> z.B. bei Zahlungen
2. Optimiert und automatisiert Geschäftsprozesse  
-> Mit Hilfe von Software direkte Datenverarbeitung  
(z.B. Check-In)
3. Erleichterte Auswertungen (Umsatz, Kursauslastung)
4. Datensicherheit  
-> Backups



## ERHEBUNG DER ANFORDERUNGEN

Kunde meldet sich im Fitnessstudio an:

1. Kunde wird von Mitarbeiter empfangen
  - > Es gibt Personal
2. Kunde entscheidet sich für ein Mitgliedschaftsmodell
  - > Es gibt Mitgliedschaftsarten (Standard/Premium)
3. Kunde füllt den Vertrag mit seinen Daten aus
  - > Ein Mitglied besitzt verschiedene Kontaktdaten
4. Kunde füllt seine Bankdaten für folgende Zahlungen aus
  - > Es gibt Zahlungsinformationen
5. Erstellung der Mitgliedschaft
  - > Es gibt Mitglieder

## ERHEBUNG DER ANFORDERUNGEN

Ein Mitglied besucht einen Kurs:

1. Ein Mitglied checkt im Fitnessstudio ein
  - > Der Check In von jedem Mitglied muss gespeichert werden
2. Der Kurs wird vorbereitet
  - > Jeder Kurs hat einen zugeordneten Raum und Trainer
3. Die Mitglieder gehen in den Kursraum
  - > Jeder Kurs hat eine bestimmte Kapazität
4. Der Kurs wird durchgeführt
  - > Jeder Kurs hat eine Start- und Endzeit



## ENTITÄTEN UND ATTRIBUTE

### **Mitglied**

MitgliedID (PK)  
Vorname, Nachname  
Geburtsdatum  
Kontaktdaten

### **Mitarbeiter**

MitarbeiterID (PK)  
Vorname, Nachname  
Geburtsdatum  
Kontaktdaten

### **CheckIn**

CheckInID (PK)  
CheckInZeit, CheckOutZeit  
MitgliedID (FK)

### **Qualifikation**

QualifikationsID (PK)  
Bezeichnung  
MitarbeiterID (FK)

### **Rolle**

RollenID (PK)  
Rollenbezeichnung  
MitarbeiterID (FK)



## ENTITÄTEN UND ATTRIBUTE

### **Mitgliedschaft**

MitgliedschaftID (PK)  
Startdatum, Enddatum  
Status (aktiv, pausiert, beendet)  
Mitgliedschaftsart  
(Standart/Premium)  
MitgliedID (FK)

### **Kurs**

KursID (PK)  
Kurstitel  
Startzeit, Endzeit  
MaxTeilnehmer  
MitarbeiterID (FK)  
RaumID (FK)

### **Zahlung**

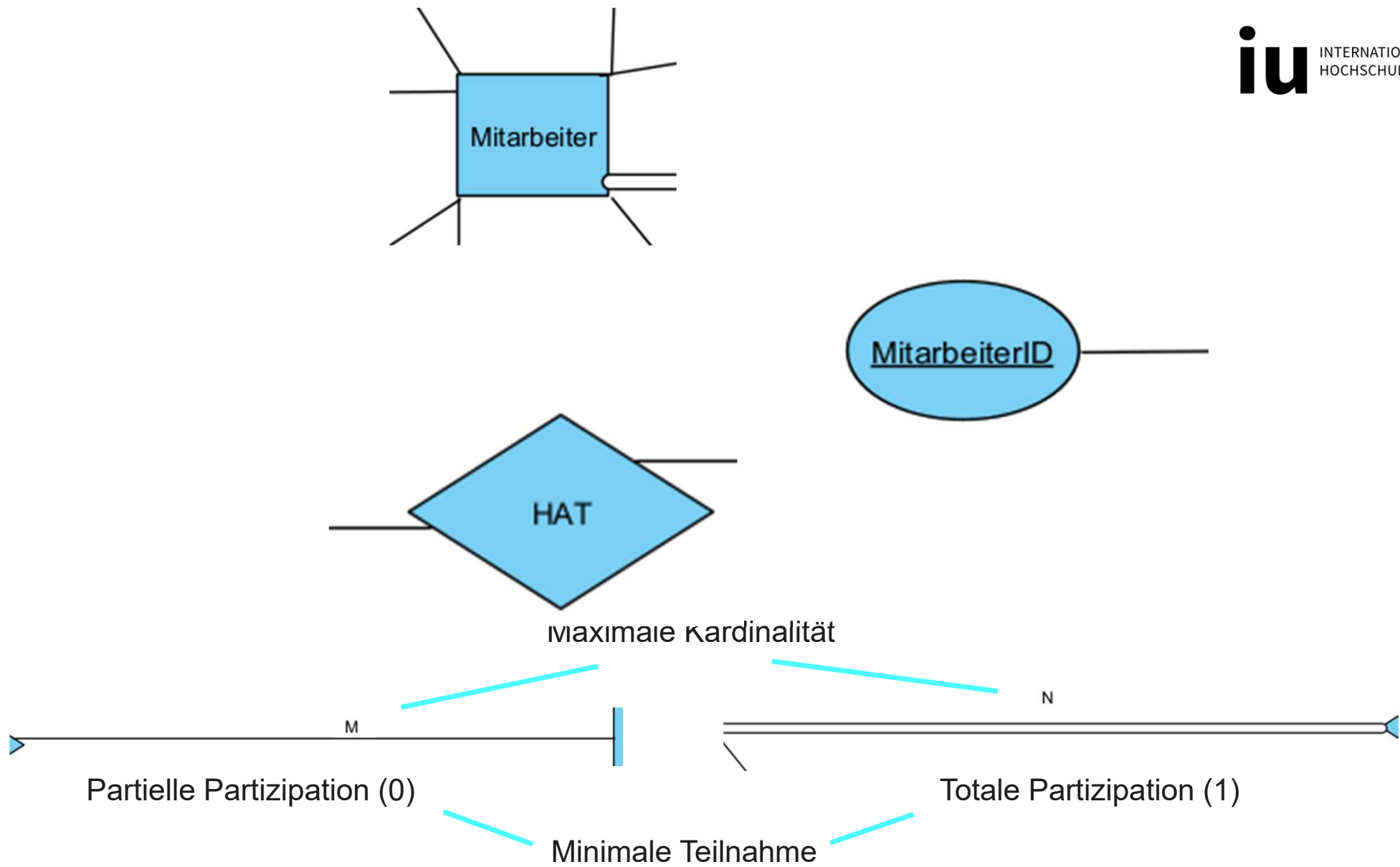
ZahlungID (PK)  
Betrag  
Zahlungsdatum  
MitgliedID (FK)

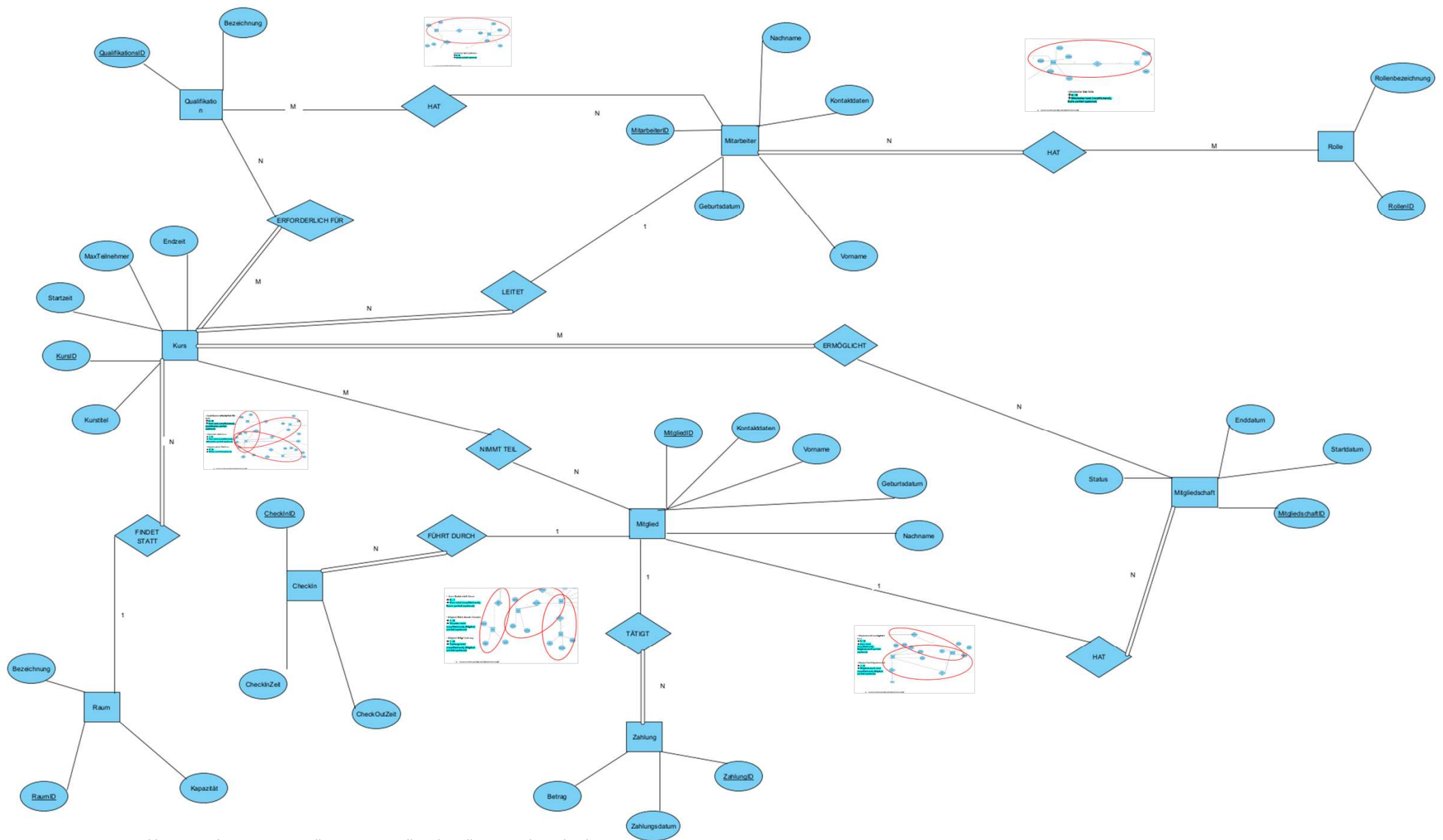
### **Raum**

RaumID (PK)  
Bezeichnung  
Kapazität  
Kurs-ID (FK)

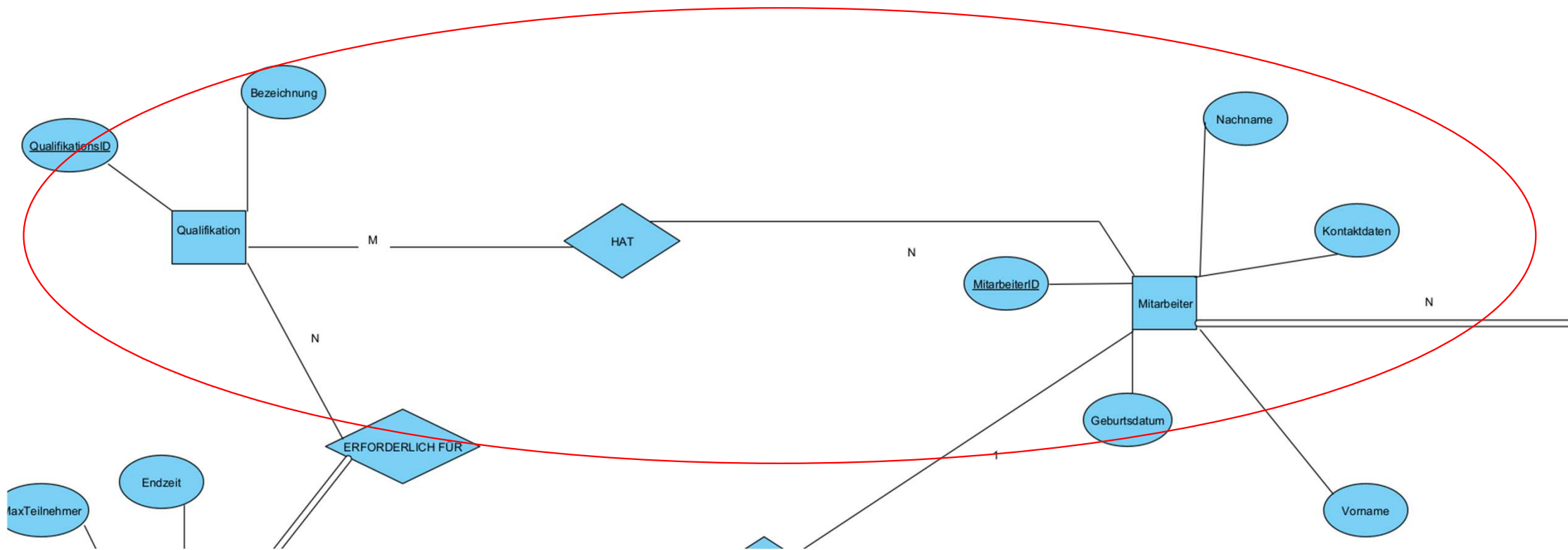
# 02

## KONZEPTIONELLES MODELL (ERD)

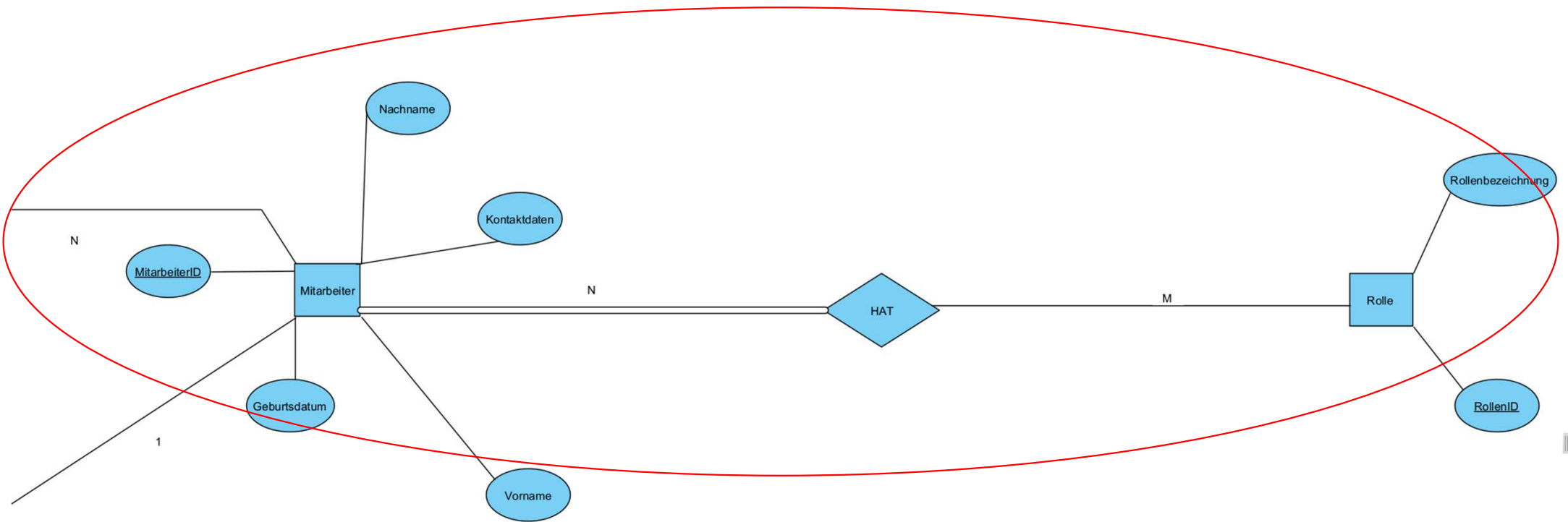




12 Abb. 1 ERD – Chen Notation, Quelle: Eigene Darstellung (erstellt mit Visual Paradigm)



- Mitarbeiter **hat** Qualifikation
- **N : M**
- **Beides partiell (optional)**



- Mitarbeiter **hat** Rolle
  - **N : M**
  - **Mitarbeiter total (verpflichtend), Rolle partiell (optional)**

- Qualifikation **erforderlich** für Kurs

→ **N : M**

→ **Kurs total (verpflichtend), Qualifikation partiell (optional)**

- Mitarbeiter **leitet** Kurs

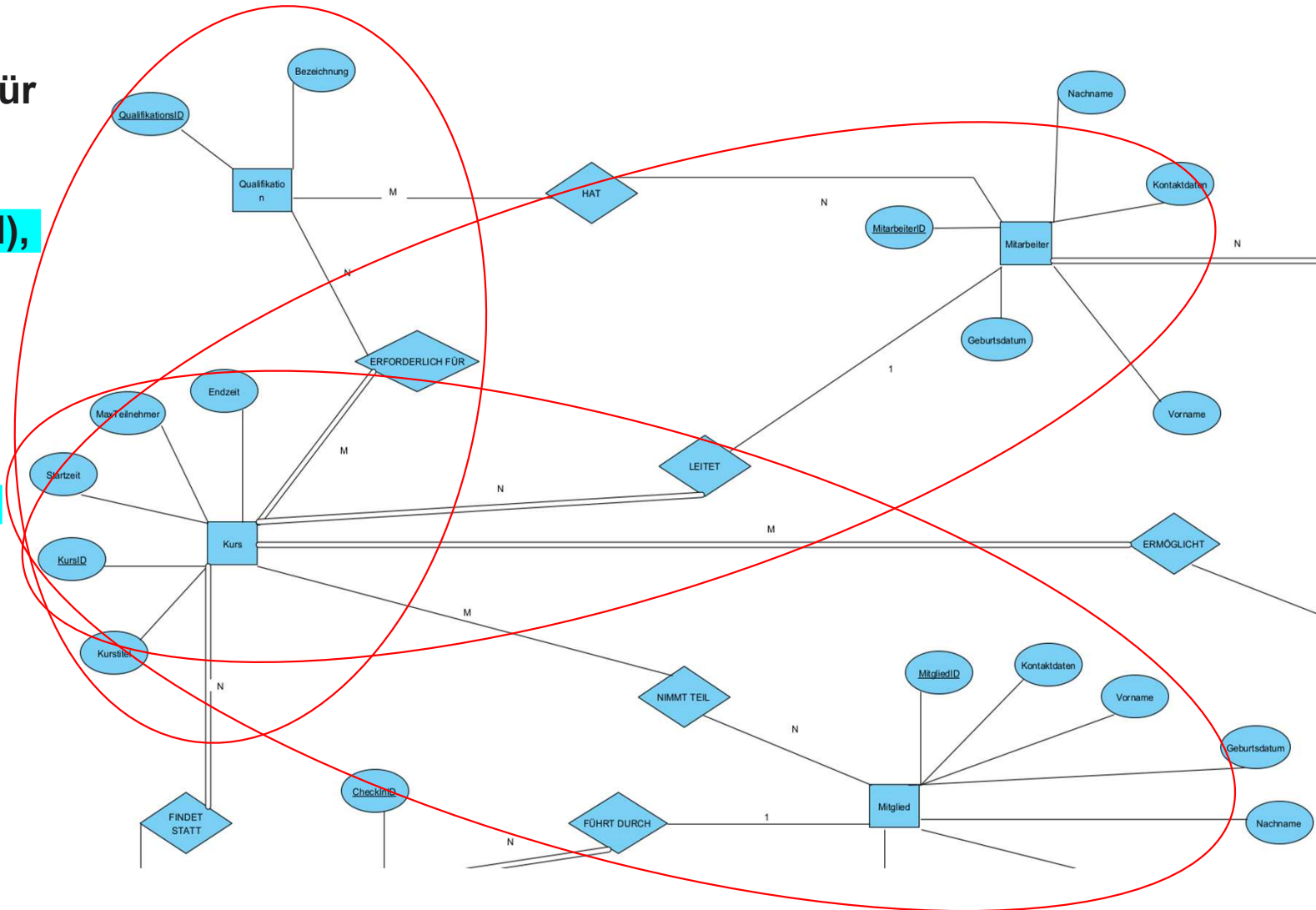
→ **1 : N**

→ **Kurs total (verpflichtend), Mitarbeiter partiell (optional)**

- Mitglied **nimmt Teil** Kurs

→ **N : M**

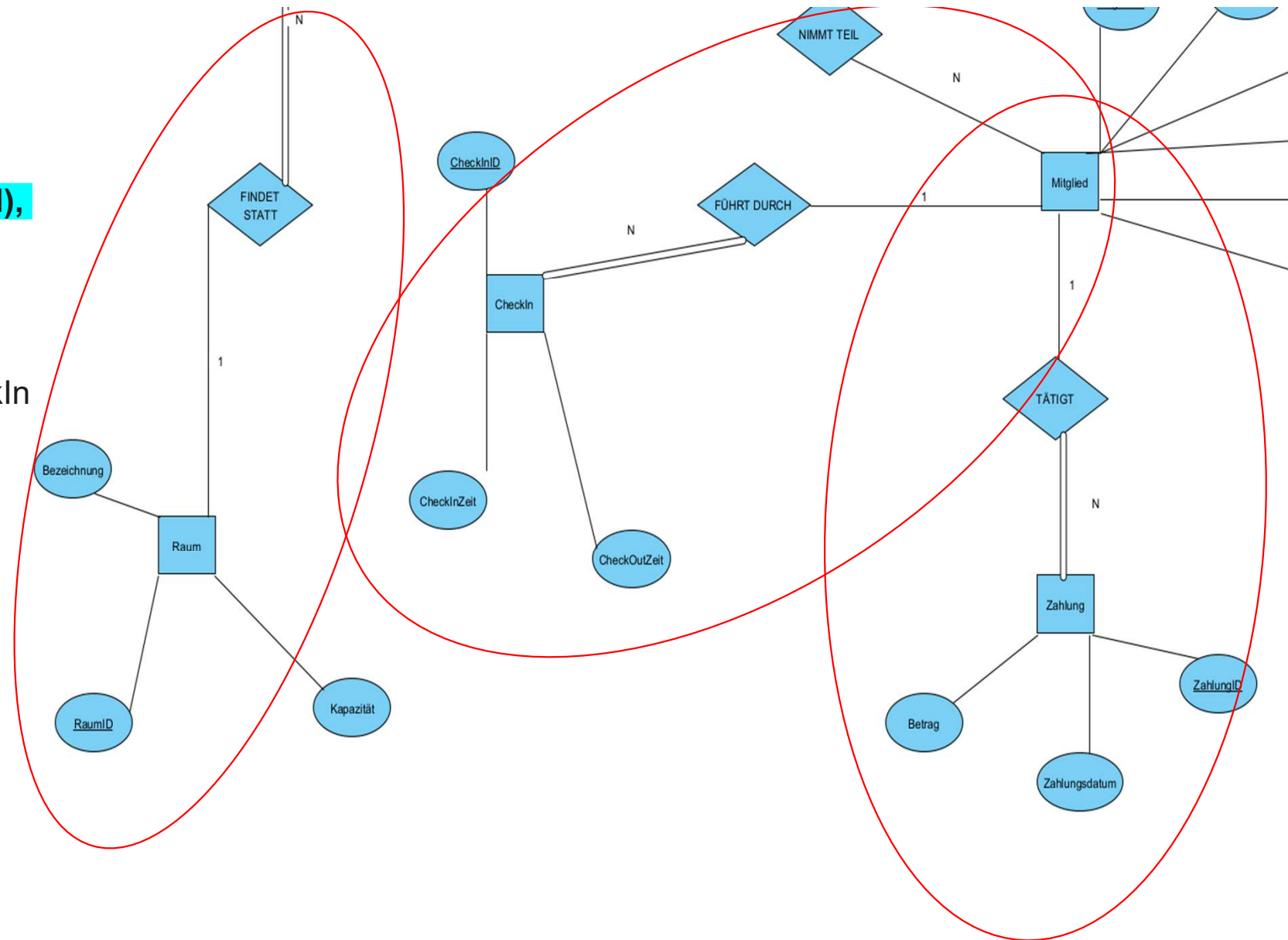
→ **Beides partiell (optional)**



- Kurs **findet statt** Raum  
 → **N : 1**  
 → **Kurs total (verpflichtend),**  
**Raum partiell (optional)**

- Mitglied **führt durch** CheckIn  
 → **1 : N**  
 → **CheckIn total**  
**(verpflichtend), Mitglied**  
**partiell (optional)**

- Mitglied **tätigt** Zahlung  
 → **1 : N**  
 → **Zahlung total**  
**(verpflichtend), Mitglied**  
**partiell (optional)**





• Mitgliedschaft **ermöglicht**  
Kurs

→ **N : M**

→ **Kurs total**

**(verpflichtend),**

**Mitgliedschaft partiell**

**(optional)**

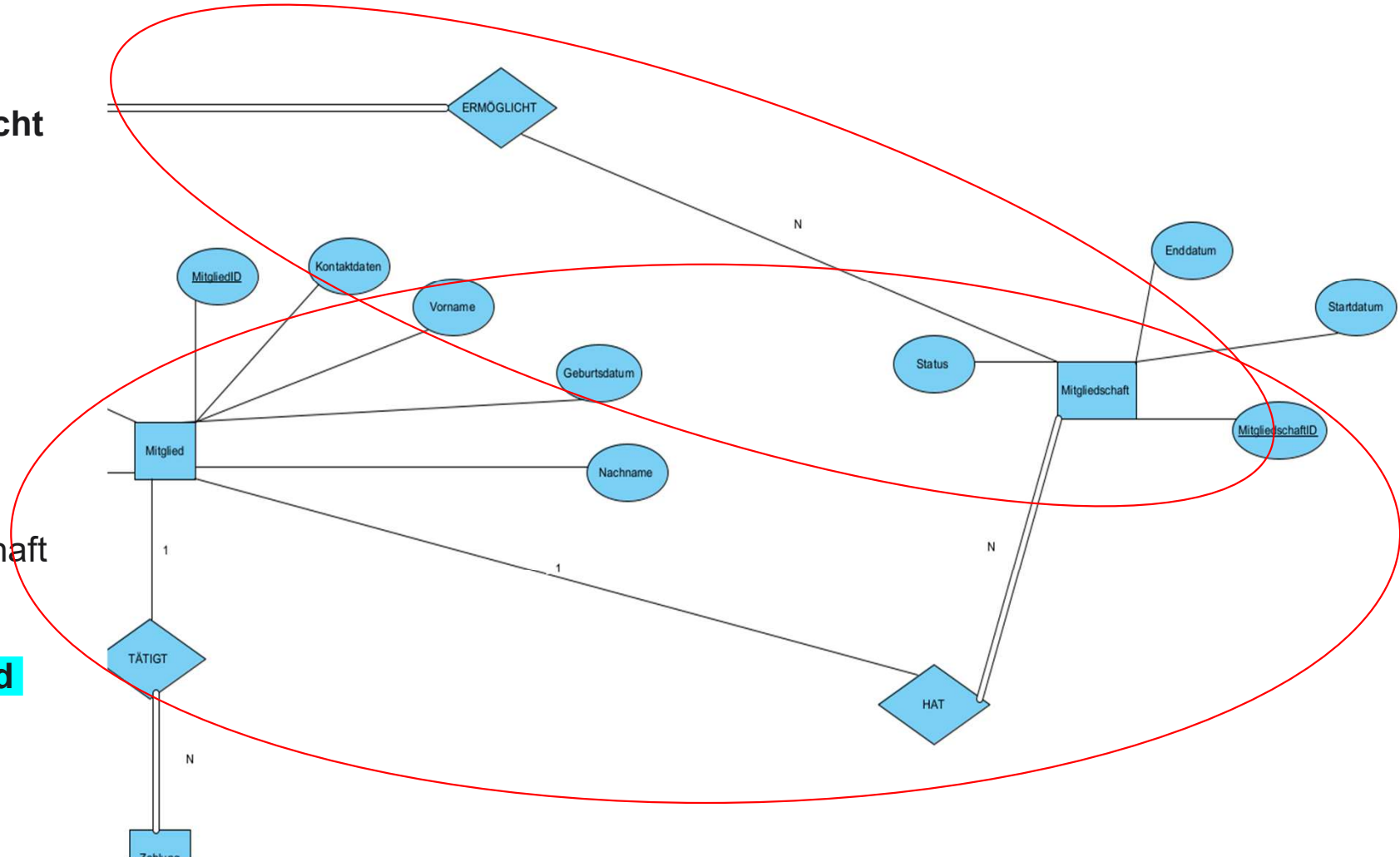
• Mitglied **hat** Mitgliedschaft

→ **1 : N**

→ **Mitgliedschaft total**

**(verpflichtend), Mitglied**

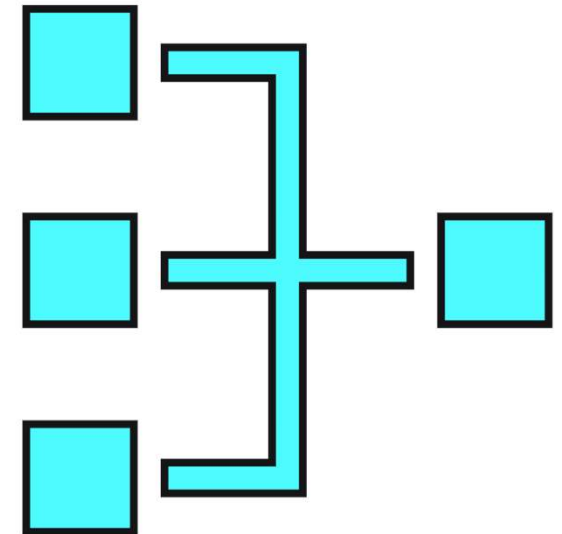
**partiell (optional)**



## KARDINALITÄTEN

- Mitglied **hat** Mitgliedschaft  
→ **1 : N**
- Mitglied **tätigt** Zahlung  
→ **1 : N**
- Mitglied **führt durch** CheckIn  
→ **1 : N**
- Mitarbeiter **leitet** Kurs  
→ **1 : N**
- Kurs **findet statt** Raum  
→ **N : 1**

- Mitarbeiter **hat** Qualifikation  
→ **N : M**
- Mitarbeiter **hat** Rolle  
→ **N : M**
- Qualifikation **erforderlich für** Kurs  
→ **N : M**
- Mitglied **nimmt Teil** Kurs  
→ **N : M**
- Mitgliedschaft **ermöglicht** Kurs  
→ **N : M**



# FAZIT



ERD bildet die zentralen  
Entitäten, Attribute und  
Beziehungen des  
Fitnessstudios ab



Kardinalitäten und  
Partizipationen sorgen für  
realistische und konsistente  
Datenstrukturen



Diagramm zeigt übersichtlich,  
wie die Entitäten  
zusammenhängen und welche  
Abhängigkeiten bestehen



Grundlage für die Umsetzung  
in das physikalische  
Datenmodell

# 03

## PHYSIKALISCHE DATENMODELLIERUNG

# ATTRIBUTTYPEN

## • Zeitstempel als Pflichtfeld

- termin.startzeit DATETIME NOT NULL (Check endzeit > startzeit)
  - YYYY-MM-DD HH:MM:SS

## • Enum / Domain (Wertebereich)

- termin\_teilnahme.status  
teilnahme\_status NOT NULL
  - Werte: bsp.  
gebucht/anwesend/abwesend

## • Unnatürlicher Primärschlüssel

- person.person\_id INT PK  
AUTO-INCREMENT
  - technische ID, unabhängig von Fachdaten, stabil für Referenzen

## • PK=FK in Subclass (Joined Subclass Key)

- mitglied.person\_id INT PK und gleichzeitig FK -> person.person\_id
  - Erzwingt „pro Person max. 1 Mitglied“

SS2

## • Geldbetrag mit Präzision

- zahlung.betrag NUMERIC(10,2)  
NOT NULL (Check betrag > 0)
  - Maximale Anzahl an Dezimalstellen in Beträgen

## Slide 21

---

**SS1** Checks können erst in der Umsetzung eingebaut werden (Checks Transaktionen)

Silas Siragusano, 2026-02-10T09:58:39.148

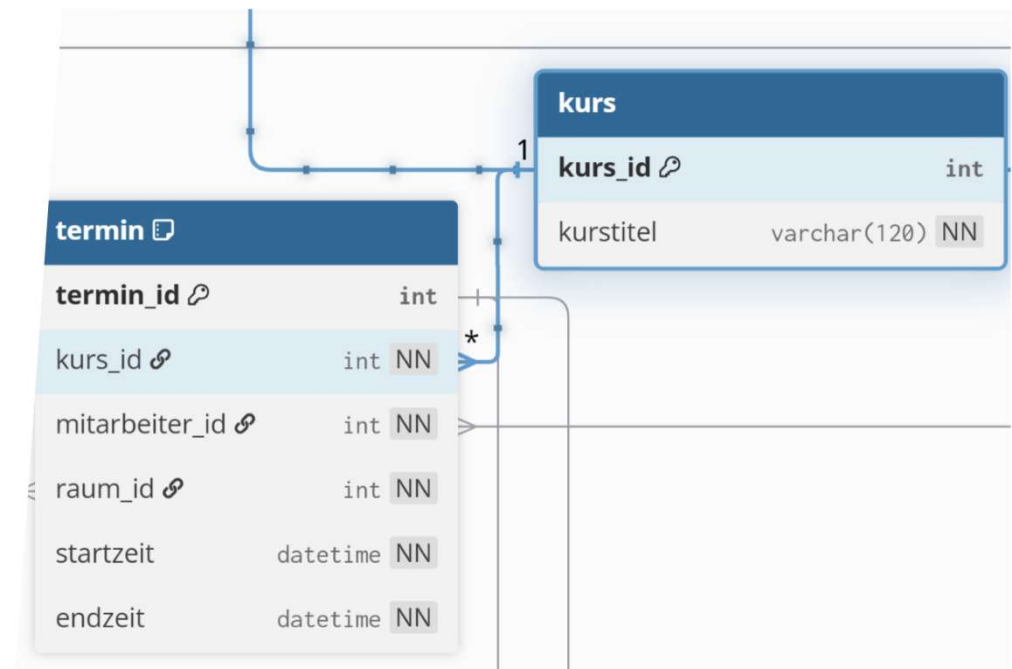
**SS2** PK=FK erzwingt 1:0..1 pro Subclass. Ob eine Person gleichzeitig Mitglied und Mitarbeiter sein darf, ist eine Business-Regel; das XOR ist mit Standard-Constraints nicht sauber über zwei Tabellen erzwingbar. Muss dann in der Applikation oder per trigger festgehalten werden.

Silas Siragusano, 2026-02-10T10:01:54.814

## Kardinalität: One-to-Many (1:N)

### 1 : N – Kurs → Termin

- **Kurs** = Format (z.B. Yoga), **Termin** = konkrete Session (Datum/Uhrzeit)
- **1:N**: Ein Kurs hat viele Termine
- Umsetzung im PDM: Fremdschlüssel liegt auf der N-Seite  
→ termin.kurs\_id (FK, NOT NULL) referenziert kurs.kurs\_id.
- Effekt: Jeder Termin gehört genau zu **einem** Kurs, aber ein Kurs kann **beliebig viele** Termine haben.

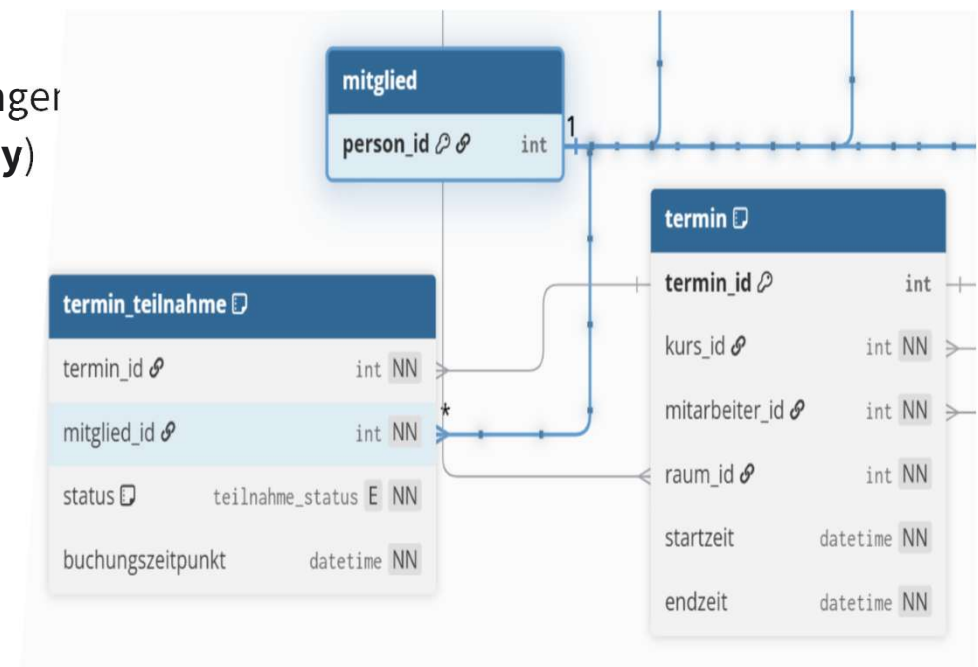


## Kardinalität: Many-to-Many (N:M)

### N : M – Mitglied ↔ Termin über Junction-Tabelle

- Fachlich: Viele Mitglieder können an vielen Terminen teilnehmen → N:M.
- Umsetzung im PDM: N:M wird als **Junction-Tabelle** modelliert → termin\_teilnahme verbindet mitglied und termin.
- **Bedeutung:** 1 Zeile = 1 Buchung/Teilnahme
- UNIQUE(termin\_id, mitglied\_id) verhindert Doppelbuchungen desselben Mitglieds für denselben Termin. (**Composite Key**)
- termin\_teilnahme bringt als **assoziative Entität** eigene Attribute mit

SS2





**SS1** Vorteil: Beziehung hat eigene Attribute:  
→ status (Enum), buchungszeitpunkt (datetime) = Infos zur Teilnahme selbst.  
Silas Siragusano, 2026-02-10T10:31:32.706

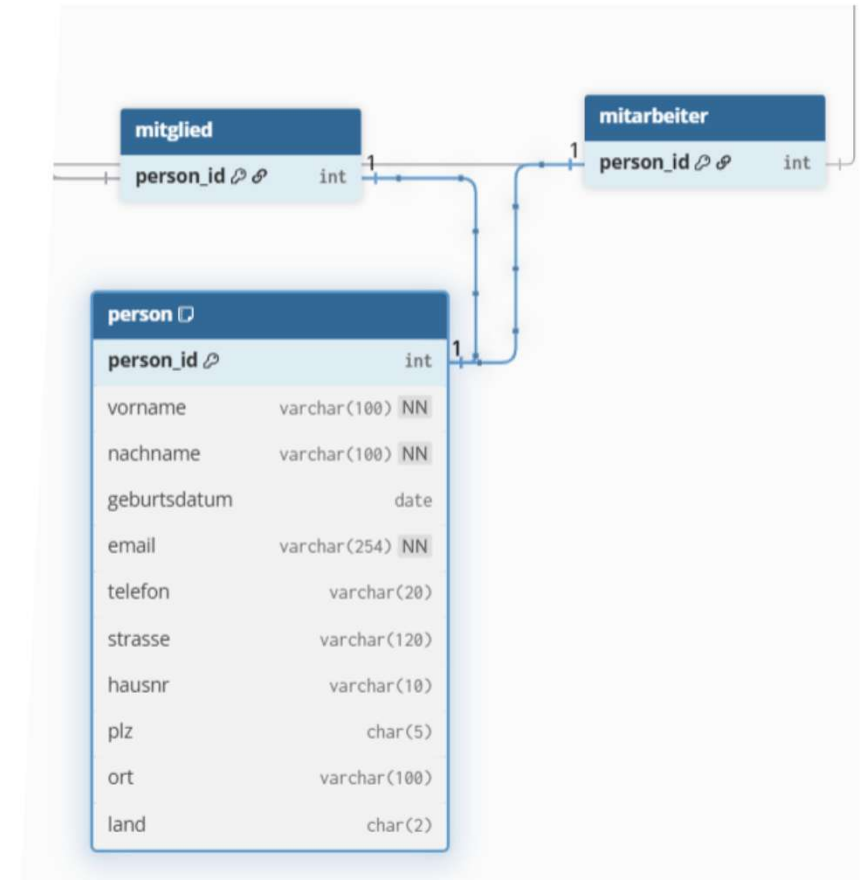
**SS2** Struktur:  
termin\_teilnahme.termin\_id (FK, NOT NULL) → termin.termin\_id  
termin\_teilnahme.mitglied\_id (FK, NOT NULL) → mitglied.person\_id  
Silas Siragusano, 2026-02-10T10:32:15.832

## 1:1-IMPLEMENTIERUNG: PK=FK (JOINED SUBCLASS)

### Joined Subclasses (Person → Mitglied / Mitarbeiter)

SS3

- person = Supertyp mit gemeinsamen Stammdaten (Name, Kontakt, Adresse)
- Mitglied und Mitarbeiter sind **Subtypen**.
- mitglied / mitarbeiter = Subtypen (**PK=FK** auf person\_id)
- Vorteil: Kontaktdaten nur einmal → **weniger Redundanz** / bessere Qualität
- Wichtiger Hinweis: Das Modell erlaubt, dass eine Person beides sein kann (Mitglied und Mitarbeiter). „Entweder-oder“ wäre eine zusätzliche **Business-Regel**.



## Slide 24

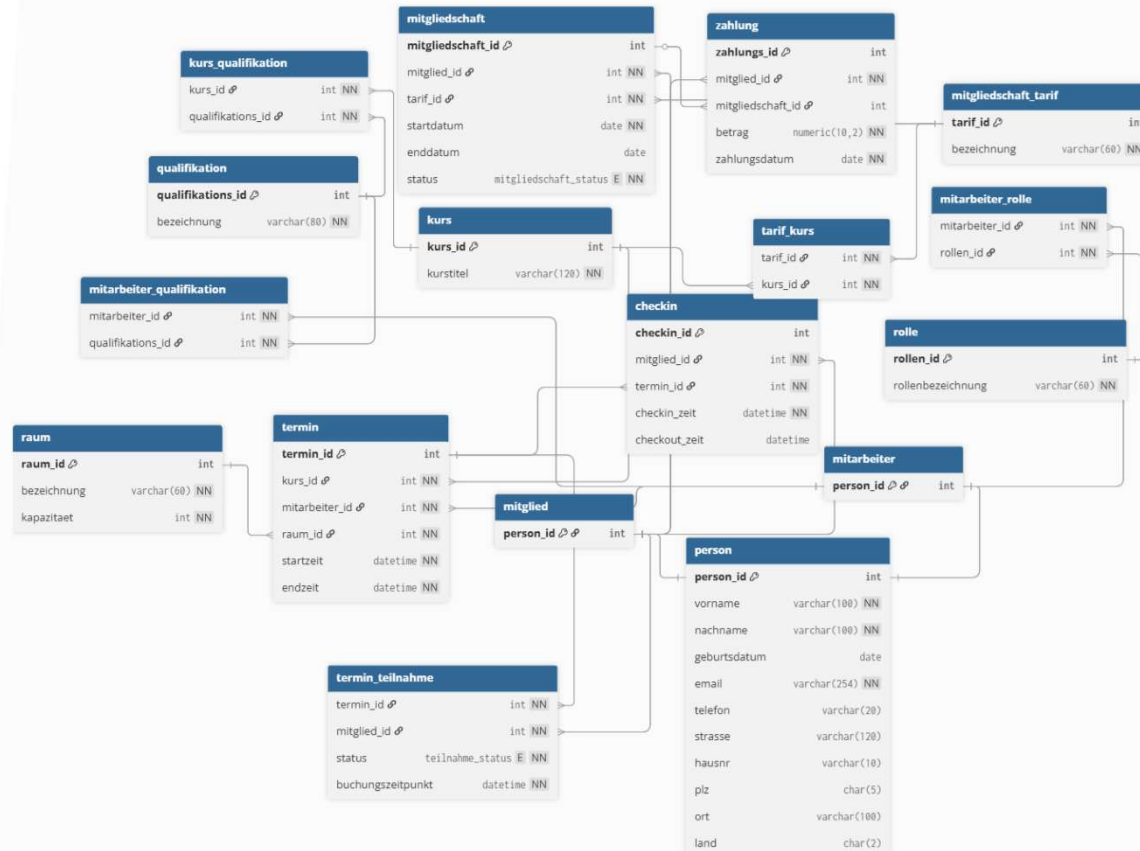
---

- SS1** Umsetzung im PDM: mitglied.person\_id und mitarbeiter.person\_id sind PK und gleichzeitig FK auf person.person\_id (PK = FK).  
Silas Siragusano, 2026-02-10T10:37:55.163
- SS2** Aufgrund der Darstellung: 1 zu 1 eigentlich sollte es (0..1): Eine Person ist nur dann Mitglied/Mitarbeiter, wenn in der jeweiligen Subtype-Tabelle eine Zeile existiert.  
Silas Siragusano, 2026-02-10T10:41:43.236
- SS3** Mitglied und Mitarbeiter sind Subtypen: sie erweitern Person um rollen-/spezifische Daten (bei euch aktuell keine Extra-Attribute, aber Struktur steht).  
Silas Siragusano, 2026-02-10T10:41:56.533
- SS4** „Unser Modell erlaubt beides; wenn man XOR will, ist das eine Business-Regel und müsste anders modelliert oder per Trigger erzwungen werden.“  
Silas Siragusano, 2026-02-10T10:44:33.522
- SS5** „PK=FK erzwingt 1:1 pro existierender Subclass-Zeile; die Optionalität (0..1) entsteht durch das optionale Vorhandensein der Subclass-Zeile.“  
Silas Siragusano, 2026-02-10T10:47:52.853

# Fazit (Physikalisches Datenmodell)

## Was kam im Gegensatz zum ERD dazu?

- Datentypen – INT, DATE, DATETIME, NUMERIC
- Längen – VARCHAR(254), CHAR(5)
- PK=FK – Joined Subclasses (Person→Mitglied/Mitarbeiter)
- NOT NULL – Pflichtfelder
- UNIQUE – keine Dubletten
- ENUM/Domain – Status-Wertebereich
- Junction-Tabellen – N:M-Auflösung (z.B. Teilnahme)
- CHECK/Validierung – einfache Plausibilität (Ende>Start, Betrag>0)
- Composite Keys für Junction Tables



# 04

## IMPLEMENTIERUNG IN MYSQL

# Datenbank erstellen

```
CREATE TABLE `mitgliedschaft` (  
  `mitgliedschaft_id` int PRIMARY KEY AUTO_INCREMENT,  
  `mitglied_id` int NOT NULL,  
  `tarif_id` int NOT NULL,  
  `startdatum` date,  
  `enddatum` date,  
  `status` ENUM ('a'  
  
  CONSTRAINT check_enddatum  
    CHECK (enddatum  
);
```

```
CREATE INDEX `mitgl  
CREATE INDEX `mitgl  
CREATE INDEX `mitgl  
CREATE INDEX `mitgl
```

```
CREATE TABLE `person` (  
  `person_id` int PRIMARY KEY AUTO_INCREMENT,  
  `vorname` varchar(100) NOT NULL,  
  `nachname` varchar(100) NOT NULL,  
  `geburtsdatum` date,  
  `email` varchar(254) UNIQUE NOT NULL,  
  `telefon` varchar(20),  
  `strasse` varchar(120),  
  `hausnr` varchar(10),  
  `plz` char(5),  
  `ort` varchar(100),  
  `land` char(52),  
  
  CONSTRAINT check_person_plz  
    CHECK (plz IS NULL OR plz REGEXP '^[0-9]{5}$')  
);
```

```
ALTER TABLE `mitgliedschaft` ADD FOREIGN KEY ( `mitglied_id` ) REFERENCES `mitglied` ( `mitglied_id` );  
ALTER TABLE `mitgliedschaft` ADD FOREIGN KEY ( `tarif_id` ) REFERENCES `mitgliedschaft_tarif` ( `tarif_id` );
```

- AUTO\_INCREMENT:
  - Einzigartige, stetig steigende ID
- NOT NULL:
  - Muss einen Wert haben
- DEFAULT:
  - Wenn kein Wert zugewiesen
- CONSTRAINT:
  - Bedingung -> Datenintegrität
- INDEX:
  - Effizientere SELECTs
- FOREIGN KEY:
  - Verweis auf PK anderer Tabelle
- UNIQUE:
  - Einzigartig

# Daten einfügen

```
INSERT INTO mitgliedschaft (mitglied_id, tarif_id, startdatum, enddatum, status)
VALUES
(1,2,'2026-01-01',NULL,'aktiv'),
(2,1,'2026-02-01',NULL,'aktiv'),
(5,2,'2006-06-07',NULL,'aktiv');
```

```
INSERT INTO mitgliedschaft (mitglied_id, tarif_id)
VALUES
(6,1),
(7,2);
```

| mitgliedschaft_id | mitglied_id | tarif_id | startdatum | enddatum | status |
|-------------------|-------------|----------|------------|----------|--------|
| 1                 | 1           | 2        | 2026-01-01 | NULL     | aktiv  |
| 2                 | 2           | 1        | 2026-02-01 | NULL     | aktiv  |
| 3                 | 5           | 2        | 2006-06-07 | NULL     | aktiv  |
| 4                 | 6           | 1        | 2026-02-11 | NULL     | aktiv  |
| 5                 | 7           | 2        | 2026-02-11 | NULL     | aktiv  |

```
INSERT INTO mitgliedschaft (mitglied_id, tarif_id, startdatum, enddatum, status)
VALUES
(1,2,'2026-01-01','2025-01-01','aktiv');
```

MySQL said: 

#4025 - CONSTRAINT `check\_mitgliedschaft\_datum` failed for `gym`.`mitgliedschaft`

# Prozedur Daten einfügen

```
CREATE PROCEDURE person_anlegen (  
    IN p_vorname VARCHAR(100),  
    IN p_nachname VARCHAR(100),  
    IN p_geburtsdatum DATE,  
    IN p_email VARCHAR(254),  
    IN p_telefon VARCHAR(20),  
    IN p_strasse VARCHAR(120),  
    IN p_hausnr VARCHAR(10),  
    IN p_plz CHAR(5),  
    IN p_ort VARCHAR(100),  
    IN p_land CHAR(2),  
    IN p_typ ENUM('mitglied', 'mitarbeiter'),  
    IN p_tarif_id INT,  
    IN p_rolle_id INT  
)
```

```
INSERT INTO person (  
    vorname, nachname, geburtsdatum, email, telefon,  
    strasse, hausnr, plz, ort, land  
)  
VALUES (  
    p_vorname, p_nachname, p_geburtsdatum, p_email, p_telefon,  
    p_strasse, p_hausnr, p_plz, p_ort, p_land  
);
```

```
DECLARE v_person_id INT;  
SET v_person_id = LAST_INSERT_ID();
```

```
SET v_person_id = LAST_INSERT_ID();  
  
IF p_typ = 'mitglied' THEN  
  
    INSERT INTO mitglied (mitglied_id)  
    VALUES (v_person_id);  
  
    INSERT INTO mitgliedschaft (  
        mitglied_id,  
        tarif_id  
    )  
    VALUES (  
        v_person_id,  
        p_tarif_id  
    );  
  
ELSEIF p_typ = 'mitarbeiter' THEN
```

```
CALL person_anlegen(  
    'Max',  
    'Mustermann',  
    '1995-05-10',  
    'max@mail.de',  
    '0170123456',  
    'Hauptstrasse',  
    '10',  
    '12345',  
    'Berlin',  
    'DE',  
    'mitglied',  
    1,  
    NULL -- Nur bei Mitarbeitern  
);
```



# 05

## **DATEENAUSWERTUNG UND TRANSAKTIONCASE MIT SQL**

# BASISABFRAGEN & SUBQUERIES: FILTERUNG AKTIVER MITGLIEDER

## SELECT

- Definiert gezielt die Spalten (Attribute), die wir auslesen wollen (z. B. vorname)

## FROM

- Bestimmt die Quelltable aus, welche wir auslesen wollen (z. B. person)

## WHERE

- Filtert Datensätze basierend auf Bedingungen (z. B. Ort oder Datum)

## DISTINCT

- Eliminiert doppelte Datensätze im Ergebnis (z. B. für eine Liste aller Wohnorte)

## Subqueris

- Verschachtelte SELECT-Statements innerhalb einer Hauptabfrage zur dynamischen Filterung

## Beispiel: Kontaktliste aller aktiven Mitglieder

```
SELECT DISTINCT vorname, nachname, email
```

```
FROM person
```

```
WHERE person_id IN (
    SELECT mitglied_id
    FROM mitgliedschaft
    WHERE status = 'aktiv'
);
```

## Ergebnis:

| vorname | nachname   | email                |
|---------|------------|----------------------|
| Max     | Mustermann | max@example.de       |
| Henrik  | Schmidt    | h.schmidt@example.de |
| Fritz   | Hanssen    | fri@example.de       |
| Aleman  | Alemania   | ger@example.de       |

# RELATIONALE VERKNÜPFUNG (JOINS)

## INNER JOIN

- Verknüpft zwei Tabellen zu einer neuen, virtuellen Tabelle

## Verbundkriterium (ON)

- Prüft Attribute auf Gleichheit (meist Primär- und Fremdschlüssel)

## AS (Aliasing)

- Gibt Tabellen kurze Namen (z. B. p für person), um den Code lesbar zu machen

## Beispiel: Verknüpfung von Person, Kursbuchung und Termindetails

```
SELECT p.nachname AS Name, k.kurstitel AS Kurs, t.startzeit  
FROM person AS p
```

```
INNER JOIN termin_teilnahme AS tt ON p.person_id = tt.mitglied_id  
INNER JOIN termin AS t ON tt.termin_id = t.termin_id  
INNER JOIN kurs AS k ON t.kurs_id = k.kurs_id;
```

## Ergebnis:

| Name      | Kurs         | startzeit           |
|-----------|--------------|---------------------|
| Hanssen   | Morning Yoga | 2026-03-01 09:00:00 |
| Alemania  | Morning Yoga | 2026-03-01 09:00:00 |
| TerStegen | Morning Yoga | 2026-03-01 09:00:00 |

# ANALYSE & AGGREGATION: AUSWERTUNG DER KURSAUSLASTUNG

## GROUP BY

- Gruppiert Datensätze mit identischen Werten, um Kennzahlen zu berechnen

## Aggregatfunktionen

- Verdichten Werte zusammen:
- **COUNT(\*)** zum Zählen
- **AVG()** für den Durchschnitt

## ORDER BY

- Sortiert das fertige Analyseergebnis nach festgelegten Kriterien:

## Reihenfolgenindikatoren

- **ASC** = Aufsteigend (A-Z 0-9)
- **DESC** = Absteigend (Z-A 9-0)

## Having

- Filtert die Ergebnisse nach der Gruppierung
- Unterscheidet sich zu Where

## Beispiel: Ranking der Kursauslastung (nur Kurse mit mehr als 2 Buchungen)

```
SELECT k.kurstitel, COUNT(tt.mitglied_id) AS Teilnehmerzahl
FROM kurs k
JOIN termin t ON k.kurs_id = t.kurs_id
JOIN termin_teilnahme tt ON t.termin_id = tt.termin_id
GROUP BY k.kurstitel
HAVING Teilnehmerzahl > 2
ORDER BY Teilnehmerzahl DESC;
```

## Ergebnis:

| kurstitel       | Teilnehmerzahl |
|-----------------|----------------|
| Spinning Power  | 6              |
| Freies Training | 4              |
| Morning Yoga    | 3              |

# TRANSAKTIONSCASE UND DATENSICHERHEIT

## Transaktion

- Eine logische Einheit von Datenbankoperationen, die dem "Alles-oder-Nichts"-Prinzip folgt

## ACID-Prinzip

- Atomarität (**Atomicity**): Eine Transaktion wird unteilbar ausgeführt
- Konsistenz (**Consistency**): Erhalt eines gültigen Datenbankzustands
- Isoliertheit (**Isolation**): Keine gegenseitige Beeinflussung paralleler Transaktionen
- Dauerhaftigkeit (**Durability**): Dauerhafte Speicherung auch bei Systemausfällen

## Steuerbefehle:

- **START TRANSACTION**: Beginn des Prozesses
- **COMMIT**: Dauerhaftes Speichern aller Änderungen.
- **ROLLBACK**: Rückgängigmachen aller Änderungen bei einem Fehler

## Beispiel: Sicherer Buchungsvorgang für ein Mitglied:

```
START TRANSACTION;
```

```
-- Wir fügen ein Mitglied hinzu
```

```
INSERT INTO termin_teilnahme (termin_id, mitglied_id, status, buchungszeitpunkt)  
VALUES (1, 1, 'gebucht', NOW());
```

```
-- JETZT PRÜFEN:
```

```
SELECT * FROM termin_teilnahme WHERE termin_id = 1 AND mitglied_id = 1;
```

```
-- Wir entscheiden uns gegen das Speichern (Simulierter Fehler)
```

```
ROLLBACK;
```

**DANKE FÜR EURE  
AUFMERKSAMKEIT!**

## QUELLENVERZEICHNIS

- Q1:** Creately. (o. J.). *Entity Relationship Modell (ER-Diagramm): Definition, Symbole, Erstellung und Vorlagen*. Abgerufen am 20. Januar 2026, von [Entity Relationship Modell | ER Diagramm Leitfaden: Symbole, Vorlagen, ...](#)
- Q2:** DBML. (o. J.). *DBML – Full Syntax Docs*. Abgerufen am 10. Februar 2026, von <https://dbml.dbdiagram.io/docs/>
- Q3:** dbdiagram.io. (o. J.). *dbdiagram.io – Database Relationship Diagrams Design Tool*. Abgerufen am 10. Februar 2026, von <https://dbdiagram.io/>
- Q4:** Ausbildung in der IT. (2026, 6. Januar). *Kardinalität (Datenbank) – 1:1, 1:n, n:m Beziehungen erklärt*. Abgerufen am 10. Februar 2026, von <https://ausbildung-in-der-it.de/lexikon/kardinalitaet>